Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №3
на тему
«ИДЕНТИФИКАЦИЯ И АУТЕНТИФИКАЦИЯ ПОЛЬЗОВАТЕЛЕЙ. ПРОТОКОЛ KERBEROS.»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Теоретические основы протокола Kerberos 4
2.2 Архитектура протокола 4
2.3 Процесс аутентификации 4
2.4 Описание реализации 5
2.5 Результаты работы программы 5
Вывод 7
Приложение А (Обязательное) Листинг программного кода 8

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является получение практических навыков в области реализации протоколов идентификации и аутентификации пользователей путём разработки симулятора протокола Kerberos. Изучение данного протокола позволяет сформировать понимание современных механизмов защищённой аутентификации в распределённых системах, основанных на симметричной криптографии и доверенной третьей стороне.

Задачи лабораторной работы включают:

– Изучение теоретических основ протокола Kerberos и принципов его работы

– Реализацию основных компонентов протокола: сервера аутентификации (*AS*), сервера выдачи билетов (*TGS*) и прикладного сервиса

– Имплементацию процесса обмена сообщениями между клиентом и серверами

– Применение криптографических примитивов для защиты передаваемых данных

– Демонстрацию полного цикла аутентификации пользователя

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Теоретические основы протокола Kerberos

Kerberos — это сетевой протокол аутентификации, разработанный в Массачусетском технологическом институте (MIT) для обеспечения безопасной идентификации пользователей в незащищённых сетях. Протокол основан на концепции доверенной третьей стороны и использует симметричное шифрование для защиты передаваемых данных. Основное преимущество Kerberos заключается в том, что пароли никогда не передаются по сети в открытом виде, а аутентификация производится с помощью криптографических билетов.

Название протокола происходит от имени трёхголового пса Цербера из греческой мифологии, что символически отражает трёхстороннюю архитектуру системы: клиент, сервер аутентификации и целевой сервис.

2.2 Архитектура протокола

Протокол Kerberos включает следующие основные компоненты:

Клиент – пользователь или процесс, который требует доступа к защищённому ресурсу. Клиент знает свой пароль, из которого формируется криптографический ключ.

Сервер аутентификации (*Authentication Service, AS*) – центральный компонент, который проверяет подлинность клиента на основе его учётных данных и выдаёт билет для получения доступа к серверу выдачи билетов.

Сервер выдачи билетов (*Ticket Granting Service, TGS*) – промежуточный сервер, который на основании билета, полученного от *AS*, выдаёт клиенту билеты для доступа к конкретным прикладным сервисам.

Прикладной сервис (*Application Service*) – целевой сервис, к которому клиент хочет получить доступ. Сервис проверяет предоставленный клиентом билет и, в случае успешной проверки, предоставляет запрашиваемые ресурсы.

2.3 Процесс аутентификации

Процесс аутентификации в протоколе Kerberos состоит из трёх основных этапов обмена сообщениями:

Этап 1: Обмен с сервером аутентификации

Клиент отправляет на *AS* запрос, содержащий своё имя и имя целевого сервиса (*TGS*). Сервер аутентификации проверяет наличие клиента в базе данных и генерирует два сообщения:

1 Билет для получения билетов (*TGT*), зашифрованный секретным ключом *TGS*, содержащий имя клиента, имя сервиса, сеансовый ключ и время истечения

2 Сеансовая информация, зашифрованная ключом клиента (производным от его пароля), содержащая сеансовый ключ для взаимодействия с *TGS*

Эти данные отправляются клиенту. Клиент расшифровывает сеансовую информацию с помощью ключа, полученного из своего пароля.

Этап 2: Обмен с сервером выдачи билетов

Клиент создаёт аутентификатор (*Authenticator*), содержащий своё имя и текущую метку времени, зашифровывает его сеансовым ключом *TGS* и отправляет на *TGS* запрос, включающий:

– Зашифрованный *TGT*, полученный от *AS*
– Зашифрованный аутентификатор
– Имя целевого прикладного сервиса

Сервер *TGS* расшифровывает *TGT* своим секретным ключом, извлекает сеансовый ключ, расшифровывает аутентификатор и проверяет соответствие имени клиента. Если проверка успешна, *TGS* генерирует:

1 Билет для прикладного сервиса (*Service Ticket*), зашифрованный ключом прикладного сервиса

2 Сеансовую информацию для клиента, зашифрованную текущим сеансовым ключом *TGS*

Эти данные отправляются клиенту.

Этап 3: Обмен с прикладным сервисом

Клиент создаёт новый аутентификатор, зашифровывает его сеансовым ключом прикладного сервиса и отправляет запрос, содержащий:

– Зашифрованный билет сервиса
– Зашифрованный аутентификатор

Прикладной сервис расшифровывает билет своим секретным ключом, извлекает сеансовый ключ, расшифровывает аутентификатор и проверяет подлинность клиента. При успешной проверке сервис предоставляет клиенту доступ к запрашиваемым ресурсам.

2.4 Описание реализации

Программа разработана на языке *Go* и реализует все четыре компонента протокола Kerberos: клиента, сервер аутентификации (*AS*), сервер выдачи билетов (*TGS*) и прикладной сервис. Для обеспечения криптографической защиты используется алгоритм *AES-256-GCM*. Исходный код программы размещён в каталоге *Lab3* и представлен в приложении к данному отчёту.

2.5 Результаты работы программы

На рисунке 2.1 представлен вывод программы, демонстрирующий успешное выполнение всех трёх этапов протокола Kerberos. Видно последовательное взаимодействие клиента с сервером аутентификации, сервером выдачи билетов и прикладным сервисом, завершающееся получением доступа к защищённым данным.

```
=== KERBEROS AUTHENTICATION PROTOCOL SIMULATION ===

[Client] Starting Kerberos authentication for alice
[Client] Target service: fileserver

--- PHASE 1: AS EXCHANGE ---
[AS] Handling authentication request from alice
[AS] Successfully issued TGT for alice
[Client] Successfully obtained TGT

--- PHASE 2: TGS EXCHANGE ---
[TGS] Handling ticket request for service: fileserver
[TGS] Successfully issued service ticket for fileserver
[Client] Successfully obtained service ticket for fileserver

--- PHASE 3: AP EXCHANGE ---
[fileserver] Received access request
[fileserver] Client alice authenticated successfully
[Client] Successfully authenticated to fileserver
[Client] Received response: Hello alice! Here is your protected data from fileserver.
```

Рисунок 2.1 – Результат выполнения программы симуляции протокола Kerberos

Программа выводит информацию о каждом этапе протокола:

– Фаза 1 показывает успешное получение *TGT* от сервера аутентификации

– Фаза 2 демонстрирует получение билета для прикладного сервиса от *TGS*

– Фаза 3 показывает успешную аутентификацию на прикладном сервисе и получение защищённых данных

Каждый компонент системы выводит соответствующие сообщения о выполняемых операциях, что позволяет проследить весь процесс аутентификации от начала до конца.

ВЫВОД

В ходе выполнения лабораторной работы разработан полнофункциональный симулятор протокола Kerberos на языке *Go*, включающий реализацию всех основных компонентов: клиента, сервера аутентификации (*AS*), сервера выдачи билетов (*TGS*) и прикладного сервиса. Реализованы все три этапа протокола аутентификации: обмен с *AS*, обмен с *TGS* и обмен с прикладным сервисом.

Изучение протокола Kerberos позволило глубже понять принципы построения систем аутентификации в распределённых средах, роль доверенной третьей стороны, механизмы защиты от перехвата паролей и повторного воспроизведения сообщений. Реализованная система демонстрирует ключевые преимущества Kerberos: однократный ввод пароля пользователем, централизованное управление аутентификацией, использование временных билетов с ограниченным сроком действия и многоуровневую защиту с помощью сеансовых ключей.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Код структур данных протокола (*types.go*)

```
package kerberos

import "time"

type Ticket struct {
  ClientName  string
  ServiceName string
  SessionKey  []byte
  Expiration  time.Time
}

type Authenticator struct {
  ClientName string
  Timestamp  time.Time
}

type ASRequest struct {
  ClientName  string
  ServiceName string
}

type ASResponse struct {
  EncryptedTGT         []byte
  EncryptedSessionPart []byte
}

type TGSRequest struct {
  EncryptedTGT           []byte
  EncryptedAuthenticator []byte
  ServiceName            string
}

type TGSResponse struct {
  EncryptedServiceTicket []byte
  EncryptedSessionPart   []byte
}

type APRequest struct {
  EncryptedServiceTicket []byte
  EncryptedAuthenticator []byte
}

type SessionPart struct {
  SessionKey  []byte
  ServiceName string
}
```

Листинг А.2 – Код криптографических функций (*crypto.go*)

```go
// Package kerberos
package kerberos

import (
  "crypto/aes"
  "crypto/cipher"
  "crypto/rand"
  "crypto/sha256"
  "fmt"
  "io"
)

func DeriveKey(password string) []byte {
  hash := sha256.Sum256([]byte(password))
  return hash[:]
}

func Encrypt(data []byte, key []byte) ([]byte, error) {
  block, err := aes.NewCipher(key)
  if err != nil {
    return nil, err
  }

  gcm, err := cipher.NewGCM(block)
  if err != nil {
    return nil, err
  }

  nonce := make([]byte, gcm.NonceSize())
  if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
    return nil, err
  }

  return gcm.Seal(nonce, nonce, data, nil), nil
}

func Decrypt(ciphertext []byte, key []byte) ([]byte, error) {
  block, err := aes.NewCipher(key)
  if err != nil {
    return nil, err
  }

  gcm, err := cipher.NewGCM(block)
  if err != nil {
    return nil, err
  }

  nonceSize := gcm.NonceSize()
  if len(ciphertext) < nonceSize {
    return nil, fmt.Errorf("ciphertext too short")
  }

  nonce, encrypted := ciphertext[:nonceSize], ciphertext[nonceSize:]
  return gcm.Open(nil, nonce, encrypted, nil)
}

func GenerateRandomKey() []byte {
```

```
  key := make([]byte, 32)
  rand.Read(key)
  return key
}
```

Листинг А.3 – Код сервера аутентификации (*as.go*)

```
package kerberos

import (
  "encoding/json"
  "fmt"
  "time"
)

type AuthenticationService struct {
  ClientKeys map[string][]byte
  TGSKey     []byte
}

func (as *AuthenticationService) HandleASRequest(req ASRequest)
(*ASResponse, error) {
    fmt.Printf("[AS] Handling authentication request from %s\n",
req.ClientName)

  clientKey, ok := as.ClientKeys[req.ClientName]
  if !ok {
    return nil, fmt.Errorf("client not found: %s", req.ClientName)
  }

  tgsSessionKey := GenerateRandomKey()

  tgt := Ticket{
    ClientName:  req.ClientName,
    ServiceName: req.ServiceName,
    SessionKey:  tgsSessionKey,
    Expiration:  time.Now().Add(8 * time.Hour),
  }

  tgtBytes, err := json.Marshal(tgt)
  if err != nil {
    return nil, fmt.Errorf("failed to marshal TGT: %v", err)
  }
  encryptedTGT, err := Encrypt(tgtBytes, as.TGSKey)
  if err != nil {
    return nil, fmt.Errorf("failed to encrypt TGT: %v", err)
  }

  sessionPart := SessionPart{
    SessionKey:  tgsSessionKey,
    ServiceName: "TGS",
  }

  sessionBytes, err := json.Marshal(sessionPart)
  if err != nil {
    return nil, fmt.Errorf("failed to marshal session part: %v", err)
```

```
  }
  encryptedSession, err := Encrypt(sessionBytes, clientKey)
  if err != nil {
    return nil, fmt.Errorf("failed to encrypt session part: %v", err)
  }

  fmt.Printf("[AS] Successfully issued TGT for %s\n", req.ClientName)

  return &ASResponse{
    EncryptedTGT:         encryptedTGT,
    EncryptedSessionPart: encryptedSession,
  }, nil
}
```

Листинг А.4 – Код сервера выдачи билетов (*tgs.go*)

```
package kerberos

import (
  "encoding/json"
  "fmt"
  "time"
)

type TicketGrantingService struct {
  TGSKey      []byte
  ServiceKeys map[string][]byte
}

func (tgs *TicketGrantingService) HandleTGSRequest(req TGSRequest)
(*TGSResponse, error) {
    fmt.Printf("[TGS] Handling ticket request for service: %s\n",
req.ServiceName)

  tgtBytes, err := Decrypt(req.EncryptedTGT, tgs.TGSKey)
  if err != nil {
    return nil, fmt.Errorf("failed to decrypt TGT: %v", err)
  }

  var tgt Ticket
  if err := json.Unmarshal(tgtBytes, &tgt); err != nil {
    return nil, fmt.Errorf("failed to unmarshal TGT: %v", err)
  }

    authenticatorBytes, err := Decrypt(req.EncryptedAuthenticator,
tgt.SessionKey)
  if err != nil {
    return nil, fmt.Errorf("failed to decrypt authenticator: %v", err)
  }

  var authenticator Authenticator
  if err := json.Unmarshal(authenticatorBytes, &authenticator); err !=
nil {
    return nil, fmt.Errorf("failed to unmarshal authenticator: %v", err)
  }
```

```
  if authenticator.ClientName != tgt.ClientName {
    return nil, fmt.Errorf("client name mismatch")
  }

  if time.Now().After(tgt.Expiration) {
    return nil, fmt.Errorf("TGT expired")
  }

  serviceKey, ok := tgs.ServiceKeys[req.ServiceName]
  if !ok {
    return nil, fmt.Errorf("unknown service: %s", req.ServiceName)
  }

  serviceSessionKey := GenerateRandomKey()

  serviceTicket := Ticket{
    ClientName:  tgt.ClientName,
    ServiceName: req.ServiceName,
    SessionKey:  serviceSessionKey,
    Expiration:  time.Now().Add(1 * time.Hour),
  }

  serviceTicketBytes, err := json.Marshal(serviceTicket)
  if err != nil {
    return nil, fmt.Errorf("failed to marshal service ticket: %v", err)
  }
  encryptedServiceTicket, err := Encrypt(serviceTicketBytes, serviceKey)
  if err != nil {
    return nil, fmt.Errorf("failed to encrypt service ticket: %v", err)
  }

  sessionPart := SessionPart{
    SessionKey:  serviceSessionKey,
    ServiceName: req.ServiceName,
  }

  sessionBytes, err := json.Marshal(sessionPart)
  if err != nil {
    return nil, fmt.Errorf("failed to marshal session part: %v", err)
  }
  encryptedSession, err := Encrypt(sessionBytes, tgt.SessionKey)
  if err != nil {
    return nil, fmt.Errorf("failed to encrypt session part: %v", err)
  }

    fmt.Printf("[TGS]  Successfully  issued  service  ticket  for  %s\n",
req.ServiceName)

  return &TGSResponse{
    EncryptedServiceTicket: encryptedServiceTicket,
    EncryptedSessionPart:   encryptedSession,
  }, nil
}
```

Листинг А.5 – Код прикладного сервиса (*service.go*)

```
package kerberos

import (
  "encoding/json"
  "fmt"
  "time"
)

type ApplicationService struct {
  Name string
  Key  []byte
}

func (s *ApplicationService) HandleRequest(req APRequest) (string,
error) {
  fmt.Printf("[%s] Received access request\n", s.Name)

  serviceTicketBytes, err := Decrypt(req.EncryptedServiceTicket, s.Key)
  if err != nil {
    return "", fmt.Errorf("failed to decrypt service ticket: %v", err)
  }

  var serviceTicket Ticket
  if err := json.Unmarshal(serviceTicketBytes, &serviceTicket); err !=
nil {
    return "", fmt.Errorf("failed to unmarshal service ticket: %v", err)
  }

    authenticatorBytes, err := Decrypt(req.EncryptedAuthenticator,
serviceTicket.SessionKey)
  if err != nil {
    return "", fmt.Errorf("failed to decrypt authenticator: %v", err)
  }

  var authenticator Authenticator
  if err := json.Unmarshal(authenticatorBytes, &authenticator); err !=
nil {
    return "", fmt.Errorf("failed to unmarshal authenticator: %v", err)
  }

  if authenticator.ClientName != serviceTicket.ClientName {
    return "", fmt.Errorf("client identity mismatch")
  }

  if time.Now().After(serviceTicket.Expiration) {
    return "", fmt.Errorf("service ticket expired")
  }

   fmt.Printf("[%s] Client %s authenticated successfully\n", s.Name,
authenticator.ClientName)

  return fmt.Sprintf("Hello %s! Here is your protected data from %s.",
authenticator.ClientName, s.Name), nil
}
```

Листинг А.6 – Код клиента (*client.go*)

```
package kerberos

import (
  "encoding/json"
  "fmt"
  "time"
)

type Client struct {
  Name     string
  Password string
}

func     (c     *Client)     Run(as     *AuthenticationService,     tgs
*TicketGrantingService, appService *ApplicationService) {
   fmt.Printf("\n[Client] Starting Kerberos authentication for %s\n",
c.Name)
  fmt.Printf("[Client] Target service: %s\n\n", appService.Name)

  clientKey := DeriveKey(c.Password)

  fmt.Println("--- PHASE 1: AS EXCHANGE ---")
  asRequest := ASRequest{
    ClientName:  c.Name,
    ServiceName: appService.Name,
  }

  asResponse, err := as.HandleASRequest(asRequest)
  if err != nil {
    fmt.Printf("[Client] AS Exchange failed: %v\n", err)
    return
  }

    sessionPartBytes,  err  :=  Decrypt(asResponse.EncryptedSessionPart,
clientKey)
  if err != nil {
    fmt.Printf("[Client] Failed to decrypt AS response: %v\n", err)
    return
  }

  var tgsSessionPart SessionPart
   if err := json.Unmarshal(sessionPartBytes, &tgsSessionPart); err !=
nil {
    fmt.Printf("[Client] Failed to parse session part: %v\n", err)
    return
  }

  fmt.Printf("[Client] Successfully obtained TGT\n\n")

  fmt.Println("--- PHASE 2: TGS EXCHANGE ---")
  tgsAuthenticator := Authenticator{
    ClientName: c.Name,
    Timestamp:  time.Now(),
  }

  tgsAuthenticatorBytes, err := json.Marshal(tgsAuthenticator)
  if err != nil {
```

```
    fmt.Printf("[Client] Failed to marshal authenticator: %v\n", err)
    return
  }
    encryptedTGSAuthenticator,  err  :=  Encrypt(tgsAuthenticatorBytes,
tgsSessionPart.SessionKey)
  if err != nil {
    fmt.Printf("[Client] Failed to encrypt authenticator: %v\n", err)
    return
  }

  tgsRequest := TGSRequest{
    EncryptedTGT:           asResponse.EncryptedTGT,
    EncryptedAuthenticator: encryptedTGSAuthenticator,
    ServiceName:            appService.Name,
  }

  tgsResponse, err := tgs.HandleTGSRequest(tgsRequest)
  if err != nil {
    fmt.Printf("[Client] TGS Exchange failed: %v\n", err)
    return
  }

                     serviceSessionPartBytes,          err          :=
Decrypt(tgsResponse.EncryptedSessionPart, tgsSessionPart.SessionKey)
  if err != nil {
    fmt.Printf("[Client] Failed to decrypt TGS response: %v\n", err)
    return
  }

  var serviceSessionPart SessionPart
          if     err     :=     json.Unmarshal(serviceSessionPartBytes,
&serviceSessionPart); err != nil {
   fmt.Printf("[Client] Failed to parse service session part: %v\n", err)
    return
  }

  fmt.Printf("[Client] Successfully obtained service ticket for %s\n\n",
appService.Name)

  fmt.Println("--- PHASE 3: AP EXCHANGE ---")
  serviceAuthenticator := Authenticator{
    ClientName: c.Name,
    Timestamp:  time.Now(),
  }

  serviceAuthenticatorBytes, err := json.Marshal(serviceAuthenticator)
  if err != nil {
   fmt.Printf("[Client] Failed to marshal service authenticator: %v\n",
err)
    return
  }
                  encryptedServiceAuthenticator,         err         :=
Encrypt(serviceAuthenticatorBytes, serviceSessionPart.SessionKey)
  if err != nil {
   fmt.Printf("[Client] Failed to encrypt service authenticator: %v\n",
err)
    return
```

```
  }

  apRequest := APRequest{
    EncryptedServiceTicket: tgsResponse.EncryptedServiceTicket,
    EncryptedAuthenticator: encryptedServiceAuthenticator,
  }

  response, err := appService.HandleRequest(apRequest)
  if err != nil {
    fmt.Printf("[Client] Service access failed: %v\n", err)
    return
  }

      fmt.Printf("[Client]   Successfully   authenticated   to   %s\n",
appService.Name)
  fmt.Printf("[Client] Received response: %s\n\n", response)
}
```

Листинг А.7 – Главный модуль программы (*main.go*)

```
package main

import (
  "fmt"

  "kerberos-go/kerberos"
)

func main() {
  fmt.Println("=== KERBEROS AUTHENTICATION PROTOCOL SIMULATION ===")

  clientName := "Dmitry"
  clientPassword := "password123"
  serviceName := "fileserver"

  clientKey := kerberos.DeriveKey(clientPassword)
  tgsKey := kerberos.GenerateRandomKey()
  serviceKey := kerberos.GenerateRandomKey()

  authenticationService := &kerberos.AuthenticationService{
    ClientKeys: map[string][]byte{
      clientName: clientKey,
    },
    TGSKey: tgsKey,
  }

  ticketGrantingService := &kerberos.TicketGrantingService{
    TGSKey: tgsKey,
    ServiceKeys: map[string][]byte{
      serviceName: serviceKey,
    },
  }

  applicationService := &kerberos.ApplicationService{
    Name: serviceName,
    Key:  serviceKey,
```

```
  }

  client := &kerberos.Client{
    Name:     clientName,
    Password: clientPassword,
  }

        client.Run(authenticationService,    ticketGrantingService,
applicationService)
}
```